

Mission 1 — Find AI Around You

Objective

To identify AI-related systems used in everyday life and understand what they do.

1. Three Everyday Systems

System	What it does	Main function
YouTube recommendations	Suggests videos based on watch history, interests, and user interactions.	Recommends
Smartphone face recognition	Analyses facial features to help identify an enrolled user and unlock the phone.	Recognises
Google Maps traffic and arrival-time prediction	Uses traffic conditions, historical patterns, and other available data to estimate travel time.	Predicts

2. Evidence That AI Is Involved

I selected YouTube recommendations as my example. YouTube explains that its recommendation system uses signals such as watch history, search history, subscriptions, likes, and feedback to personalise recommendations.

Official source: <https://support.google.com/youtube/answer/6342839>

This documentation provides evidence that YouTube uses a recommendation system based on user-related signals. It does not mean that every recommendation is produced by one particular machine-learning model.

3. Could a Rule-Based Program Produce Similar Behaviour?

A simple rule-based program could recommend videos using manually defined conditions. For example, if a user watches several VLSI videos, the program could recommend more VLSI videos.

However, a fixed rule-based program would generally be less flexible when handling complex patterns and changing user preferences. A machine-learning-based recommender can learn patterns from data and personalise recommendations more effectively.

Conclusion

AI-related features are present in many everyday applications. Recommendation systems, facial recognition, and traffic prediction demonstrate how computing systems can recommend content, recognise patterns, and estimate future outcomes. However, an application appearing intelligent does not by itself prove that it uses AI. Reliable documentation or other evidence is needed to confirm the technology involved.

Mission 2 — AI, ML, Deep Learning and Generative AI

1. Artificial Intelligence (AI)

Artificial Intelligence is a field of computing that aims to build systems capable of performing tasks that normally require human-like intelligence, such as reasoning, recognising patterns, making decisions, and solving problems.

Example: A system that helps detect suspicious transactions in online banking.

2. Machine Learning (ML)

Machine Learning is a subset of AI in which computers learn patterns from data and use those patterns to make predictions or decisions instead of relying entirely on manually written rules.

Example: An email spam filter trained using examples of spam and legitimate messages.

3. Deep Learning (DL)

Deep Learning is a subset of machine learning that uses neural networks with multiple layers to learn complex patterns from large amounts of data.

Example: A deep neural network that recognises objects in photographs.

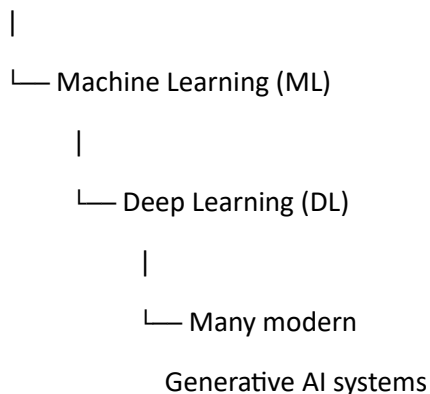
4. Generative AI (GenAI)

Generative AI refers to AI systems that generate new content, such as text, images, audio, video, or computer code, based on patterns learned from data.

Example: An AI assistant that generates a Python program from a natural-language description.

5. Relationship Between These Concepts

Artificial Intelligence (AI)



This diagram is a simplified representation. Generative AI is not limited to deep learning in every possible implementation, although modern generative AI systems commonly use deep-learning models.

6. Review Using an AI Assistant

I used an AI assistant to review the diagram and explanations. The review helped clarify that AI is the broad field, machine learning is one approach within AI, and deep learning is a type of machine learning. Many modern generative AI systems use deep-learning architectures.

I also clarified that not every AI system learns from data and not every deep-learning system generates content.

Conclusion

AI, ML, DL, and GenAI are related but different concepts. Understanding their relationship helps me identify the technology behind real-world applications instead of treating all intelligent software as the same thing.

Mission 3 — Is It Really AI?

1. Classification Table

System	Classification	Reason
Calculator	Rule-based/traditional software	Performs mathematical operations using programmed instructions.
Temperature warning rule	Rule-based/traditional software	Triggers an alert when a measured temperature crosses a predefined threshold.
Spam filter	ML-based AI, in a typical modern implementation	Can learn patterns that distinguish spam from legitimate email.
Document summariser	Generative AI, when using an LLM	Produces a shorter version of a document using a language model.
Traffic ETA prediction	ML-based AI, in a typical modern implementation	Can use historical and current traffic data to estimate arrival times.

These classifications describe typical implementations. A particular product may use a different approach, so its actual implementation should be checked when accuracy matters.

2. One Easy Classification

The calculator was easy to classify because a basic calculator follows programmed mathematical operations. It does not need to learn patterns from training data to calculate a sum or product.

For example, when I enter $5 + 3$, the calculator applies arithmetic rules and displays 8.

3. One Difficult Classification

Traffic ETA prediction was more difficult to classify because prediction does not automatically mean that a system uses machine learning.

A simple program could estimate travel time using a fixed speed and distance formula. A more advanced system could learn from historical traffic patterns and current road conditions. Therefore, the implementation must be investigated before classifying a particular traffic application.

4. My Own Example

Example: An intelligent test-result analyser for digital circuits.

A rule-based version could flag a failure whenever the actual output differs from the expected output. A machine-learning-based version could analyse historical test results and identify patterns associated with likely failures.

The first version uses explicit rules, whereas the second may learn patterns from data.

Conclusion

A system should not be classified as AI simply because it performs a useful or complicated task. Traditional software, machine learning, and generative AI can produce different types of behaviour. Understanding how a system works is more important than judging it by its output alone.

Mission 4 — Make AI Explain Itself, Then Test It

1. First Prompt

Prompt used:

"Explain how a Large Language Model (LLM) produces an answer to my prompt."

2. Beginner-Level Explanation

A Large Language Model is an AI system trained on large amounts of text and other data, depending on the model. During training, it learns statistical patterns and relationships between tokens, which are pieces of text such as words or parts of words.

When a user enters a prompt, the model processes the input as tokens and uses its learned parameters and computational layers to calculate likely next-token outputs. It generates a response by producing tokens step by step, using the prompt and the context available during generation.

The model does not simply search a database of complete answers every time. It generates text using patterns learned during training and the current context. Depending on the system, additional tools or retrieval mechanisms may also contribute information.

3. Second Prompt

Prompt used:

"Now explain how an LLM produces an answer to me as a beginner using a simple example."

4. Simple Example

Imagine asking an LLM: "The capital of India is..."

The model processes the text and uses its learned patterns and current context to generate a likely continuation, such as "New Delhi."

For a longer question, the model generates a sequence of tokens that forms a response. The result depends on the model, the prompt, the context, and the generation settings.

This example illustrates text generation; it does not mean that every response is guaranteed to be correct.

5. Verification of Important Claims

Claim 1: Language models can generate text by predicting the next token.

Verification source: Google Cloud — What are large language models (LLMs)?

<https://cloud.google.com/ai/llms>

The source describes LLMs and explains their ability to predict and generate text. This supports the basic explanation of token-by-token generation.

Claim 2: The Transformer architecture uses attention mechanisms to process relationships between tokens.

Verification source: Attention Is All You Need — original research paper.

<https://arxiv.org/abs/1706.03762>

The research paper introduces the Transformer architecture and its attention-based approach. It supports the explanation of attention as an important component of many modern language models. Not every LLM necessarily uses the same architecture.

6. What AI Explained Well

The AI explanation helped me understand that an LLM processes a prompt and generates a response based on learned patterns rather than simply retrieving a complete answer from a database.

7. What I Needed to Clarify

I needed to clarify that next-token prediction does not guarantee factual correctness. A response can be grammatically convincing while containing incorrect or unsupported information. I also clarified that tools and retrieval systems can contribute to a response when they are part of the application.

Conclusion

This investigation helped me understand the basic process of LLM text generation. It also taught me that AI explanations should be checked against reliable technical sources instead of being accepted solely because they sound convincing.

Mission 5 — Can AI Be Confidently Wrong?

1. Objective

The objective is to investigate whether an AI assistant can provide an incorrect or incomplete answer and to learn why independent verification is important.

2. Prompt

"Is 0.999... exactly equal to 1 in mathematics? Explain your answer and show a proof."

3. AI Answer to Investigate

The mathematically correct answer is yes. The repeating decimal 0.999... is exactly equal to 1, not merely approximately equal to 1.

A short proof is:

Let $x = 0.999\ldots$

Then:

$$10x = 9.999\ldots$$

Subtract the first equation from the second:

$$10x - x = 9.999\ldots - 0.999\ldots$$

$$9x = 9$$

Therefore:

$$x = 1$$

Since x represents 0.999..., we conclude that $0.999\ldots = 1$.

4. Independent Verification

Verification source: Khan Academy — repeating decimals and their relationship to fractions and real numbers.

<https://www.khanacademy.org/math/arithmetic/arith-review-decimals>

I would use the source to review repeating decimals and their mathematical representation, and compare that explanation with the algebraic proof above.

5. Verification Result

The answer is mathematically correct, as demonstrated by the algebraic proof. The proof is stronger evidence for this specific claim than simply accepting the AI's confidence.

The verification also has limits: checking one mathematical answer does not prove that every answer from the same AI system is correct.

6. Lesson Learned

AI can produce answers that sound convincing, but the quality of an answer must be judged using evidence and independent reasoning. For engineering work, I should verify important calculations, technical claims, and generated code before relying on them.

Conclusion

This investigation demonstrates the importance of verification. Rather than trusting an answer because it sounds confident, I should test it using a reliable source, mathematical reasoning, or an appropriate experiment.

Mission 6 — Chatbot or Agent?

1. Important Concepts

Large Language Model (LLM)

An LLM is a model trained to process and generate language. It can answer questions, summarise documents, explain concepts, and generate code.

Example: A language model that explains a Verilog module.

AI Application

An AI application is a software product that uses one or more AI models to perform a task. It may also include a user interface, databases, business logic, and external tools.

Example: A learning application that uses an LLM to explain digital electronics.

Retrieval-Augmented Generation (RAG)

RAG is an approach in which an application retrieves relevant information from an external knowledge source and provides it to a generative model as context for answering a question.

Example: An assistant that searches a collection of semiconductor datasheets before answering a question about a component.

Tool-Using Assistant

A tool-using assistant can call external tools or software to obtain information or perform operations instead of relying only on text generation.

Example: An assistant that runs a Python calculation and explains the result.

AI Agent

An AI agent is a system that uses a model and available tools to work toward a goal through one or more steps. Depending on its design, it may observe results, decide what to do next, and continue until it reaches a stopping condition.

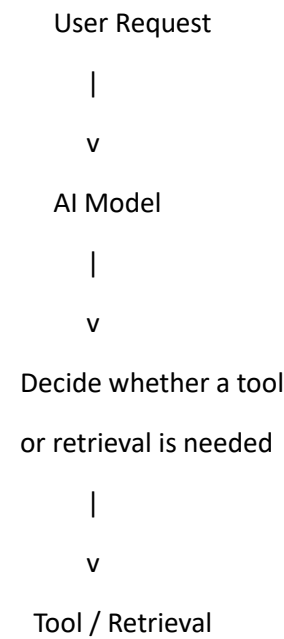
Example: An agent that searches technical documentation, compares relevant information, and prepares a report with citations.

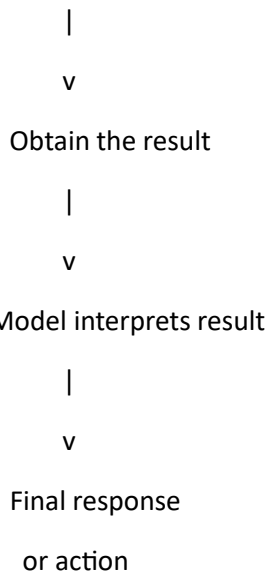
2. Comparison Table

Feature	Basic chatbot	RAG application	Tool-using assistant	AI agent
Main purpose	Respond to messages	Answer using retrieved information	Use external tools	Work toward a goal through steps
Retrieves documents	Not necessarily	Yes, as a core feature	Optional	Optional
Calls tools	Not necessarily	May use retrieval tools	Yes	Usually, when configured
Multi-step workflow	Limited or optional	Depends on design	Depends on design	Common
Example	Explaining a concept	Answering from technical manuals	Running a calculation	Researching and preparing a report

These categories can overlap. A chatbot can use RAG, and an agent can use both retrieval and external tools.

3. Workflow Diagram





A simple chatbot may answer directly without calling any external tool. A tool-using assistant can obtain additional information or perform an operation before answering.

4. Everyday Example of an Agentic Workflow

Consider a technical research assistant asked to compare three FPGA development boards.

The system could break the request into steps, retrieve official specifications for each board, compare their FPGA devices and available interfaces, identify missing information, and prepare a comparison report with source links.

A human should still check the specifications, ensure the sources are reliable, and make the final selection.

Conclusion

A chatbot primarily provides conversational responses, whereas an agentic system can carry out a sequence of actions toward a goal. RAG and tool use can improve an AI application's access to information and ability to perform tasks. These capabilities are useful, but the final result still requires verification.

Mission 7 — What Actually Runs AI?

1. What Is a CPU and What Is It Good At?

A Central Processing Unit (CPU) is a general-purpose processor that executes program instructions. It is designed to handle a wide range of tasks, including operating-system operations, program control, sequential logic, and general computation.

CPUs are useful for coordinating AI applications, running preprocessing tasks, managing memory, and executing AI inference workloads when appropriate.

2. What Is a GPU and Why Is It Useful for AI?

A Graphics Processing Unit (GPU) is a processor designed to execute many operations in parallel. GPUs contain many processing units that can perform large numbers of similar mathematical operations at the same time.

AI models often perform large matrix multiplications and tensor operations. These computations can be parallelised, making GPUs useful for training and running many AI models.

3. What Is an NPU or AI Accelerator?

A Neural Processing Unit (NPU) or AI accelerator is specialised hardware designed to execute common AI operations efficiently.

Depending on its architecture, it may support operations such as matrix multiplication, tensor processing, and neural-network inference. Specialised hardware can improve performance per watt and reduce latency for supported workloads.

Examples include NPUs in mobile devices and AI accelerators in data centres.

4. What Is Parallel Computation?

Parallel computation means performing multiple computational operations simultaneously instead of executing every operation one after another.

For example, a large matrix multiplication contains many calculations that can be distributed across multiple processing units. Parallel computation can reduce execution time when the workload and hardware support it.

However, not every task can be parallelised equally well, and communication and memory-access costs can limit performance.

5. Why Does AI Depend on Compute and Memory?

AI models require computation to perform mathematical operations and memory to store model parameters, input data, intermediate results, and outputs.

Large models may contain billions of parameters. Moving these parameters between memory and processing units can consume substantial time and energy.

Consequently, AI performance depends not only on the processor's computational capability but also on memory capacity, bandwidth, data movement, and software efficiency.

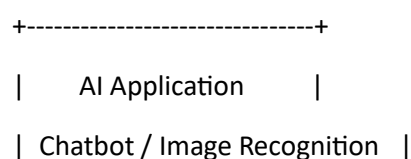
6. Difference Between Training and Inference

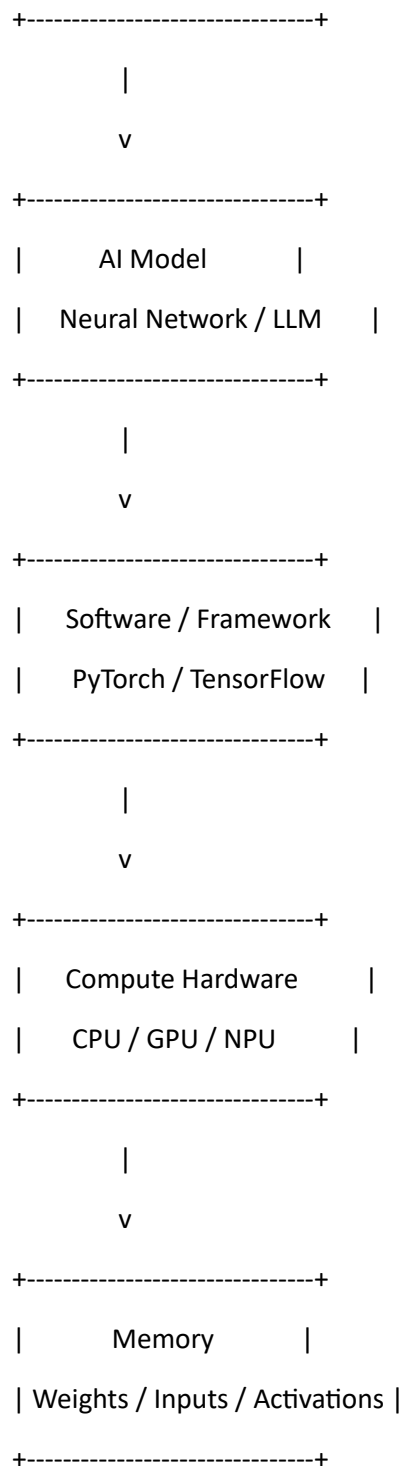
Training is the process of adjusting a model's parameters using data and an optimisation procedure. It commonly involves forward computation, calculating a loss, backpropagation, and parameter updates.

Inference is the process of using an already-trained model to produce predictions or outputs for new inputs. Inference generally does not update the model's parameters.

Training often requires substantial compute and memory resources, particularly for large models. Inference requirements depend on the model, input size, batch size, latency target, and hardware.

7. AI Hardware-Stack Diagram





The diagram is a simplified conceptual view. In a real system, the framework and operating system coordinate hardware and memory, and memory may be distributed across several levels.

8. Real AI Workload Example

Workload: Image Classification

Consider an image-classification system that identifies objects in a photograph.

The application sends an image to a trained neural network. The software framework prepares the image and executes the model's operations on available hardware. The model performs numerical calculations using its learned parameters and generates class scores.

A GPU can be useful when processing many images because convolution and matrix operations can be parallelised. A specialised AI accelerator may be suitable for efficient inference on supported operations. A CPU may also be sufficient for smaller models or low-throughput applications.

The appropriate hardware depends on the model size, performance requirements, memory capacity, power consumption, cost, and available software support.

Conclusion

AI software relies on computing hardware to execute the mathematical operations required by a model. CPUs provide general-purpose computation, GPUs support highly parallel workloads, and AI accelerators specialise in selected neural-network operations. Understanding compute, memory, training, and inference helps engineers evaluate the hardware requirements of AI applications.

Mission 8 — Where Could AI Help My VLSI Track?

1. Selected Track: Physical Design (PD)

Physical Design (PD) is a stage in the VLSI design flow where a synthesized circuit is transformed into a physical layout. It involves floorplanning, placement, clock tree synthesis, routing, and timing and physical verification.

2. Task Where AI Could Help

One task where AI could help in Physical Design is Power, Performance, and Area (PPA) optimization.

During physical design, engineers must explore different implementation options to meet timing, area, and power constraints. This process can require multiple iterations of placement, routing, and timing analysis.

AI and machine-learning techniques can help explore different design configurations, predict implementation outcomes, identify promising optimization options, and reduce the time spent on repetitive experiments.

3. AI Application Table

Physical Design Task	How AI Could Help	Why Human Engineering Knowledge Is Still Needed
Floorplanning	Help explore floorplan configurations and estimate their potential impact on area and timing.	Engineers must consider design constraints, congestion, macros, and connectivity.
Placement optimization	Help identify promising placement configurations to improve timing and reduce congestion.	Engineers must validate placement legality, wirelength, congestion, and timing.

Timing optimization	Analyse timing reports and suggest possible ways to address critical paths.	Engineers must understand setup and hold constraints, cell delays, parasitics, and timing requirements.
Routing optimization	Help predict routing congestion and identify areas that may need attention.	Engineers must verify routing design rules, signal integrity, and physical feasibility.
PPA optimization	Help explore implementation settings to balance power, performance, and area.	Engineers must check the actual results and ensure all design constraints are satisfied.

4. Benefits of Using AI in Physical Design

AI could help engineers analyse large amounts of implementation data, identify patterns in timing and congestion reports, and explore multiple design configurations more efficiently.

For example, an AI-assisted optimization system could evaluate different implementation settings and help identify configurations that may improve timing without exceeding area constraints.

AI could also assist in summarising timing reports, generating analysis scripts, and identifying potential causes of design failures.

5. Why Human Engineering Knowledge Still Matters

AI-generated recommendations are not guaranteed to produce a better physical design. A configuration that improves timing may increase area or power, and a predicted improvement may not appear in the actual implementation.

Physical Design engineers must understand floorplanning, placement, clock tree synthesis, routing, static timing analysis, design rules, and PPA trade-offs.

They must validate recommendations using the EDA tools, inspect implementation reports, and confirm that the final design satisfies all required constraints.

Conclusion

AI can support Physical Design by assisting PPA optimization, placement exploration, timing analysis, and congestion prediction. It can reduce repetitive work and help engineers explore design options more efficiently. However, actual implementation results must be verified using physical design and timing analysis tools. AI supports the optimization process, while engineers remain responsible for technical decisions and final design quality.

Mission 9 — My AI Working Agreement

Objective

To establish five rules for using AI responsibly and effectively while learning engineering concepts and solving technical problems.

Rule 1: Verify Important Technical Information

I will independently verify important AI-generated technical claims using reliable sources, official documentation, mathematical reasoning, simulations, or experiments.

For example, when AI explains a digital circuit or suggests a DFT testing approach, I will check the explanation against appropriate technical references and test results.

Rule 2: Never Share Confidential or Proprietary Information

I will not enter confidential company information, restricted design files, proprietary RTL, passwords, private keys, or other sensitive information into an AI tool unless its use is explicitly authorised under the applicable policies.

I will use public examples or appropriately sanitised data when learning or experimenting.

Rule 3: Take Responsibility for My Final Work

I will ensure that I understand the final answers, diagrams, programs, and technical decisions I submit. I will not treat AI-generated output as automatically correct or submit work that I cannot explain.

AI can assist my learning, but I remain responsible for the quality, correctness, and integrity of my work.

Rule 4: Test AI-Generated Code and Engineering Solutions

When AI generates code, scripts, circuit logic, or technical procedures, I will review and test them before relying on the results.

For example, I will compile or simulate Verilog code where appropriate, check expected outputs, and investigate any mismatches instead of assuming that generated code is correct.

Rule 5: Use AI to Improve Understanding, Not Just to Obtain Answers

I will ask AI for explanations, examples, alternative approaches, and feedback on my understanding. I will try to solve problems myself and use AI to identify gaps in my knowledge.

I will also compare important claims with independent evidence and record what I learned from any errors or corrections.

Justification for My Rules

Missions 4 and 5 emphasise that AI explanations should be checked against reliable evidence. Even when an explanation sounds convincing, it is important to distinguish a plausible response from a verified result.

This is why verification is central to my working agreement. In engineering, an incorrect technical conclusion can affect design quality, testing, reliability, and debugging. Independent checks and experiments help me decide when an AI-generated result can be trusted.

Conclusion

My working agreement is to use AI responsibly, verify important claims, protect confidential information, test generated solutions, and take ownership of my final work. These rules will help me become an engineer who can use AI effectively without depending on it blindly.

